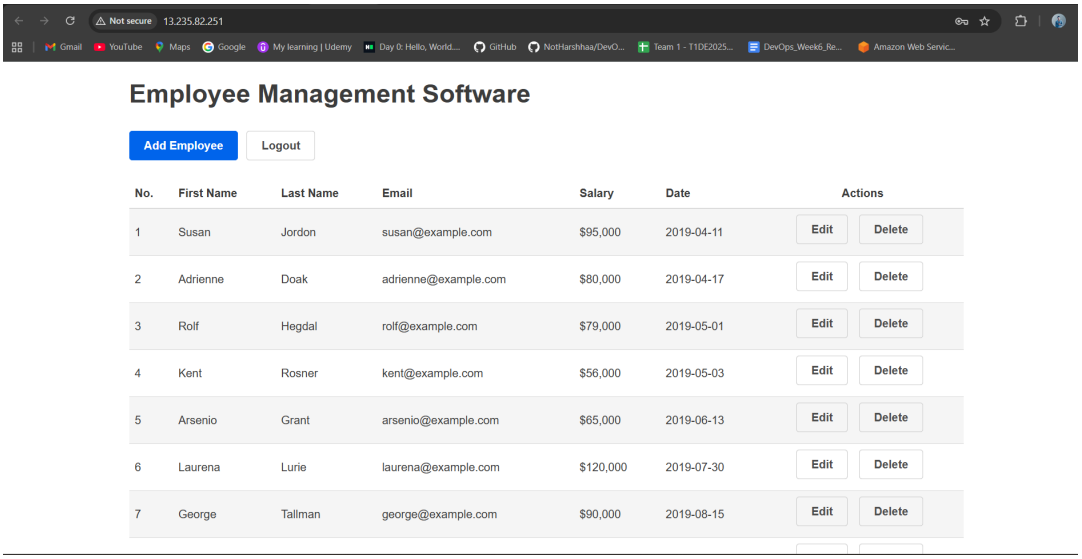
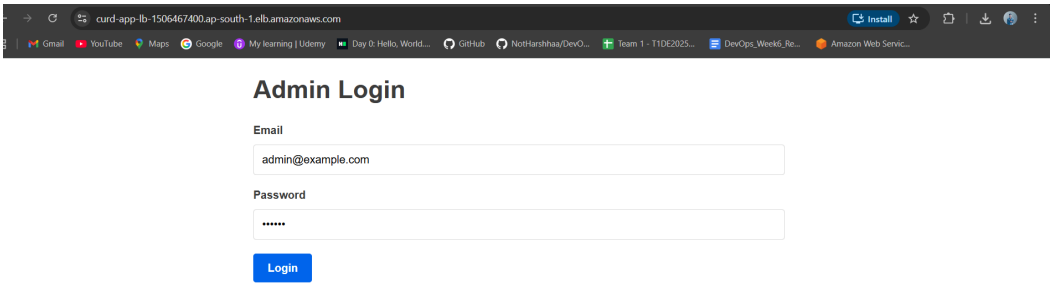
Crud-App Deployment and Metrics Retrieval With Code Scan

- **Task to be done:**
  - Deploy a react.js application in ECS fargate.
  - Connect it to https://project.cybersapient.site
  - Setup Sonarqube in EC2 and connect it with this repo so in each workflow trigger the code will be scanned.
  - Setup Grafana in EC2 and create a dashboard to see ECS cpu and memory metrics.
  - Connect the application with New Relic for performance metrics.

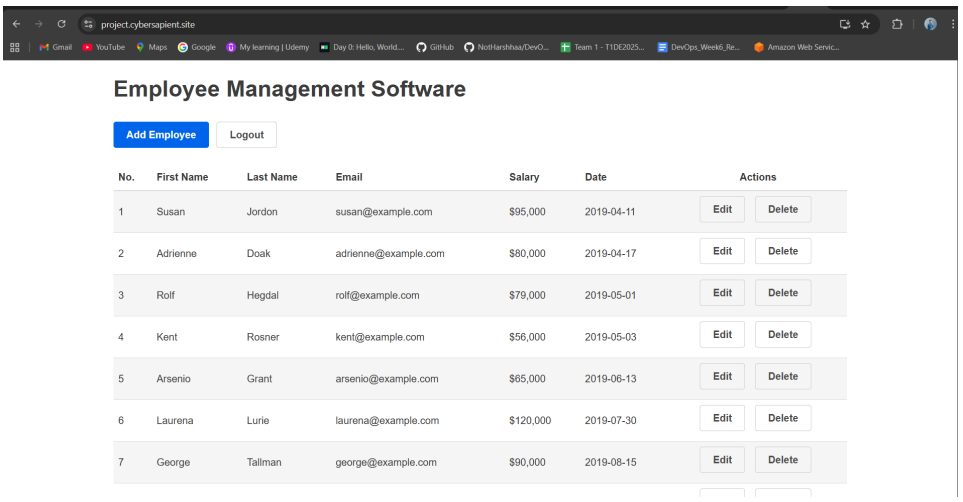
- **Setup Guide:**
  - Deploy a react.js application from a GitHub repository to AWS ECS (Fargate) using GitHub Actions, expose it publicly, and connect it to a custom subdomain (project.cybersapient.site) with proper accessibility to HTTP and HTTPS and www to non-www redirection.
  - Created a nginx.conf to fix React SPA routing as well as prevent blank pages.
  - A Dockerfile was created for react.js application.
  - The application listens on port 80.
  - Created ECR for container storage and ECS cluster, task definition and service in AWS.
  - Allow inbound 80 for ECS task security group.
  - Set port mapping to 80.
  - A build.yml workflow was created.
  - On push to main branch:
    - Checkout github
    - Run sonarqube scan on every trigger
    - Build Docker image
    - Push to ECR
    - Update ECS task definition
    - Deploy new revision to ECS service
  - GitHub Actions completed successfully and automated deployments.

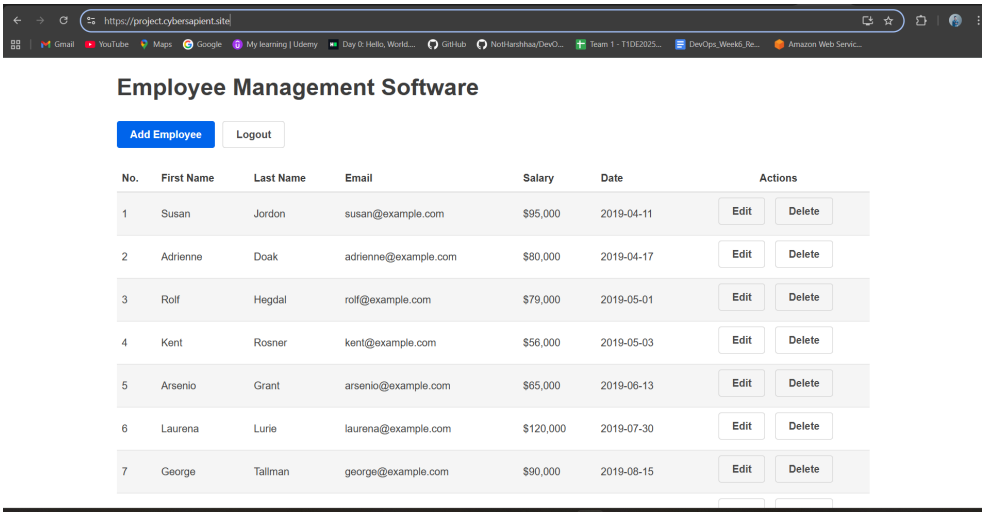


- Enabled cloudwatch insights in the ECS service.
- ACM certificate created for project.cybersapient.site and www.project.cybersapient.site.
- ALB Security Group allows inbound 80 and 443 from anywhere.
- Internet-facing Application Load Balancer created.
- Target group type: IP
- Target port: 80
- ECS service registered tasks automatically into the target group.
- Added HTTP : 80 Listener
- Redirect to HTTPS
- Added HTTPS : 443 Listener
- Added 2 rules
- Default rule redirects to the target group which has registered ECS task.
- Specified the ACM certificate generated for the domains.
- Second rule with if condition redirecting to https where host is node.cybersapient.site when host header is www.project.cybersapient.site.
- Created records **www.node** and **node** in the created hosted zone cybersapient.site. Set alias set for ALB and set the alias target to the ALB DNS name.
- Verified if the application was accessible through ALB DNS Name.



- The Application was successfully accessed when visited via www.project.cybersapient.site which redirects to https://project.cybersapient.site.



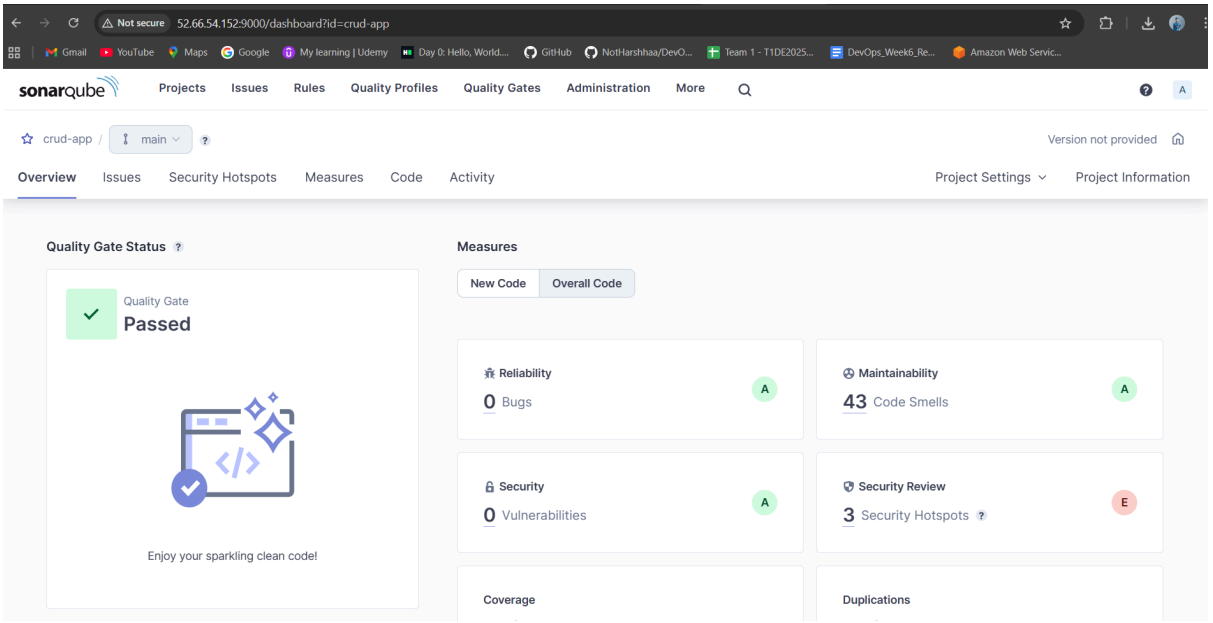


- Launched an instance for running Sonarqube. Added 9000 in the security group for sonarqube.

- Installed Sonarqube. Created a user sonar and gave ownership to /opt/sonarqube-10.3.0.82913 for running the sonar.sh.

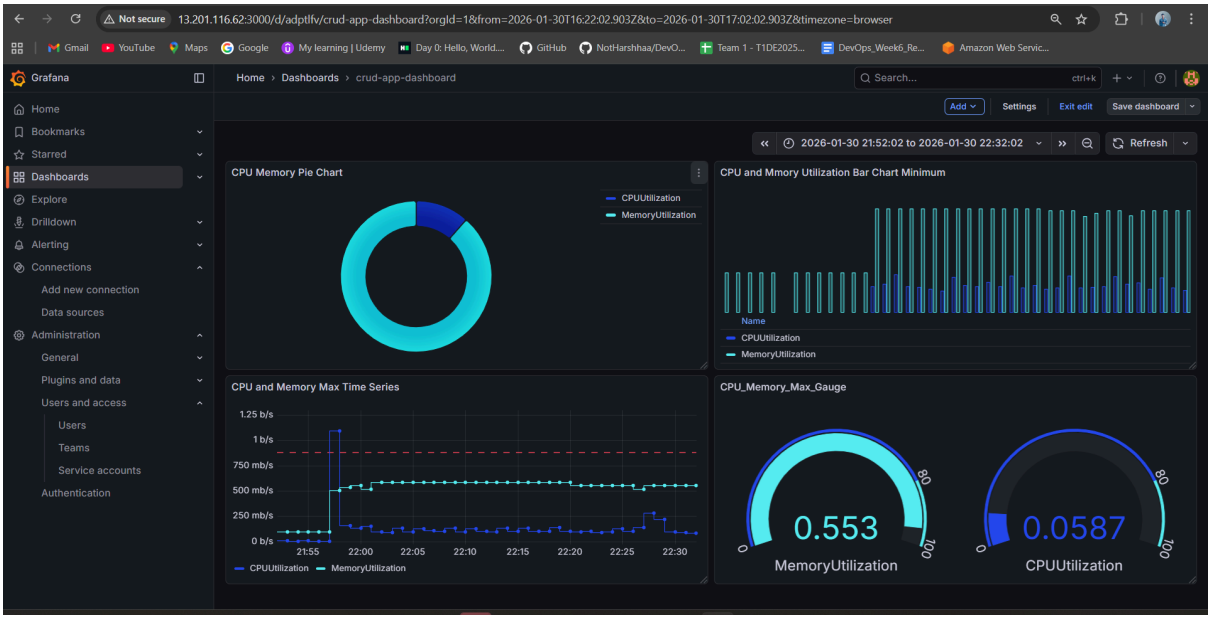
```
total 4
-rwxr-xr-x. 1 sonar sonar 101 Jan 30 11:05 elasticsearch
drwxr-xr-x. 2 sonar sonar  22 Jan 30 11:10 linux-x86-64
drwxr-xr-x. 2 sonar sonar  22 Jan 30 11:05 macosx-universal-64
drwxr-xr-x. 3 sonar sonar  63 Jan 30 11:05 windows-x86-64
drwxr-xr-x. 2 sonar sonar  25 Jan 30 11:05 winsw-license
[sonar@ip-172-31-43-140 bin]$ cd linux-x86-64/
[sonar@ip-172-31-43-140 linux-x86-64]$ ls -l
total 8
-rwxr-xr-x. 1 sonar sonar 7192 Jan 30 11:05 sonar.sh
[sonar@ip-172-31-43-140 linux-x86-64]$ ./sonar.sh start
/usr/bin/java
Starting SonarQube...
Started SonarQube.
[sonar@ip-172-31-43-140 linux-x86-64]$ ./sonar.sh status
/usr/bin/java
SonarQube is running (27180).
[sonar@ip-172-31-43-140 linux-x86-64]$ |
```

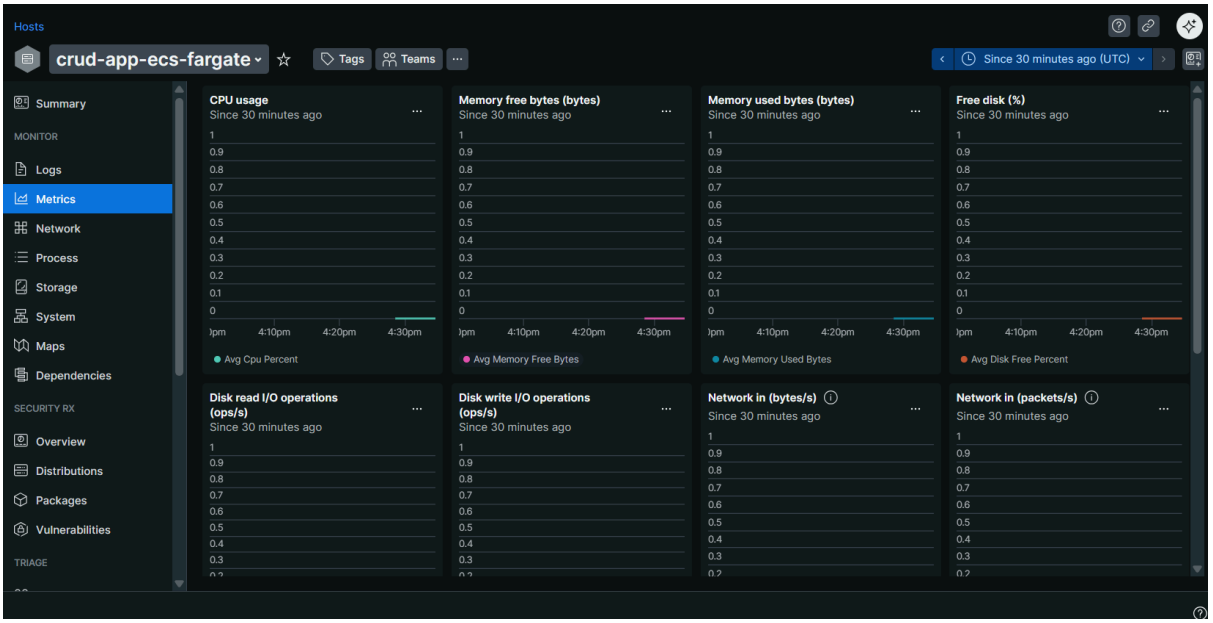
- Accessed sonarqube via http://<EC2-Public-IP>:9000.
- Created a local project and Defined Project Display Name and a Unique Project key.
- Created Sonar Token and defined the token and the sonar host url i.e., http://<EC2-Public-IP>:9000 as secrets in the github repository settings.
- Updated the build.yml and pushed and sonarqube scanned the code and results were obtained.



- Launched another instance for grafana. Added port 3000 as inbound in the security group.
- Logged into the instance and installed grafana.
  - sudo apt update && sudo apt upgrade -y
  - sudo apt install -y software-properties-common wget
  - wget -q -O - https://packages.grafana.com/gpg.key | sudo gpg --dearmor -o /usr/share/keyrings/grafana.gpg

    - sudo apt update
    - sudo apt install grafana -y
    - sudo systemctl start grafana-server
    - sudo systemctl enable grafana-server
  - Created a new user only for grafana without any AWS console access. Attached inline policy for cloudwatch read only access.
  - Generated secret key and access key for the grafana user.
  - Accessed grafana UI via http://<EC2-public-ip>:3000
  - In the connections tab, added a new data source and selected cloudwatch.
  - Specified the secret key and access key for integrating with the AWS account. Saved and tested the connection.
  - Created a Dashboard with new panels. Set cloudwatch as the data source and added queries for namespace as AWS/ECS.
  - Selected CPU utilization and memory utilization as metric name in each queries.
  - In the Dimension tab provided selected the ecs cluster and ecs service of the deployed application.
  - Created the dashboard for the Cpu and memory visualization.



  - Logged into New Relic for Metric collections. Generated Ingest-License API key and copied it.
  - Created a new task revision with updated task definition.json
  - The json defines a container which will run alongside the app as a sidecar.
  - It Collects infra & container metrics
  - Authenticates using the generated license key.
  - Sends data to New Relic

- Logs safely to CloudWatch
- Updated the ECS service to use the newly revisioned task. After the task was activated and running checked the new relic UI.
- New host with the ecs fargate display name was created in the entities.
- Metrics were successfully collected and obtained in new relic UI.